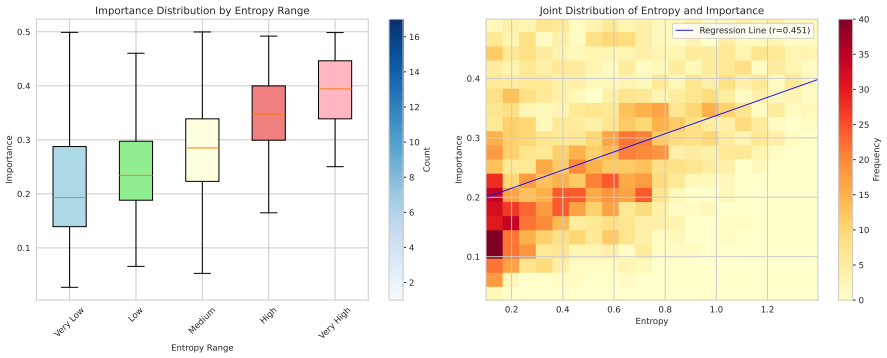


Figure 2: **Empirical validation of the relationship between token entropy and importance. (Left)** The box chart shows the average importance for tokens binned into five equal-width intervals based on their entropy value. This indicates that importance increases with entropy. **(Right)** The scatter plot displays the joint distribution of entropy and importance, with the regression line confirming a significant positive correlation (r=0.451).

Finally, the aggregated representation $\mathbf{h}$ is computed as the weighted sum of the hidden states, using the entropy-derived weights:

$$\mathbf{h} = \sum_{i=1}^{n} w_i h_i \tag{4}$$

By adaptively focusing on the most informative parts of the input prompt, EGTP provides a higher-quality feature representation for the downstream length prediction task.

3.2 Length Regression via Soft Label Distribution

Next, we will use our above feature representations to build a model for length prediction. While length prediction is a regression task, standard MSE loss is highly sensitive to outliers or heavy-tailed distributions, which have been shown in Figure 1a. The common alternative, classification via length binning, ignores the crucial concept of distance between true and predicted values. Our approach overcomes these limitations by designing a prediction head that is both robust to outliers like classification and distance-aware like regression.

Our method begins by converting the continuous length target y into a soft probability distribution $\mathbf{p}$ to serve as the ground truth label. We first discretize the target space into K predefined bins. Instead of using a one-hot vector, we generate a soft distribution where, for a true length falling into bin i, the probability p_j for any bin j is inversely proportional to its distance from bin i. This is formulated as:

$$p_j = \frac{\exp(-|j-i|)}{\sum_{k=1}^{K} \exp(-|k-i|)} \tag{5}$$

Next, using the feature vector $\mathbf{h}$ from EGTP, our model produces two concurrent outputs. The first is a **classification prediction** $\hat{\mathbf{p}}$, which is a K-dimensional probability distribution $[\hat{p}_1, \ldots, \hat{p}_K]$ obtained via a softmax layer. The second is the final **regression prediction** $\hat{y}$, which is calculated as the expected value of the predicted distribution. Assuming c_i is the center value of the i-th bin, this is computed as:

$$\hat{y} = \sum_{i=1}^{K} \hat{p}_i \cdot c_i \tag{6}$$

Finally, the model is trained by optimizing a joint loss function that combines a Cross-Entropy (CE) loss and a Mean Squared Error (MSE) loss, balanced by a hyperparameter λ:

$$\mathcal{L} = \lambda \mathcal{L}_{\text{CE}}(\mathbf{p}, \hat{\mathbf{p}}) + (1 - \lambda)\mathcal{L}_{\text{MSE}}(y, \hat{y}) \tag{7}$$

The $\mathcal{L}_{\text{CE}}$ term encourages the predicted distribution $\hat{\mathbf{p}}$ to align with the soft label distribution $\mathbf{p}$, thereby providing stable, gradient-friendly supervision. Simultaneously, the $\mathcal{L}_{\text{MSE}}$ term directly minimizes the error between the final continuous prediction $\hat{y}$ and the true length y, ensuring regression accuracy.

3.3 Progressive Length Prediction (PLP)

In scenarios such as online reinforcement learning, a system often generates multiple candidate responses with varying lengths from a single prompt. In this context, a static, pre-generation prediction is insufficient. To address this, we introduce **Progressive Length Prediction (PLP)**. PLP leverages the autoregressive nature of LLMs by making a new prediction at **each decoding step**. At timestep t, its objective is to predict the *remaining* number of tokens to be generated $y_{\text{rem}}^{(t)}$. This is done to leverage the information from all previously generated tokens to make a more accurate prediction at each step.

To do this, PLP first forms a dynamic input representation z_t by combining the prompt feature vector $\mathbf{h}$ with the hidden states of the already generated tokens $\{h'_1, \ldots, h'_t\}$:

$$z_t = \text{Aggregate}(\mathbf{h}, \{h'_1, \ldots, h'_t\}) \tag{8}$$

where Aggregate$(\cdot)$ is a simple concatenation function. This representation z_t is then passed through the same prediction head described in Section 3.2 to yield the final prediction for the remaining length, $\hat{y}_{\text{rem}}^{(t)}$.

The model is trained by minimizing the average loss across all timesteps. The total loss for a single sequence is:

$$\mathcal{L}_{\text{PLP}} = \frac{1}{T} \sum_{t=1}^{T} \mathcal{L}(y_{\text{rem}}^{(t)}, \hat{y}_{\text{rem}}^{(t)}) \tag{9}$$

where T is the total sequence length and $\mathcal{L}$ is the joint loss function defined in Eq. (7). By iteratively refining its forecast, PLP enables dynamic adjustments to scheduling strategies, thereby improving resource utilization.

4 Experiments

4.1 Dataset construction

To evaluate our proposed method in complex settings, we constructed the **ForeLen**, which comprises two core scenarios designed to comprehensively assess predictor performance under challenging conditions.

Scenario 1: Long-Sequence and Complex Reasoning Generation. This scenario focuses on long-text and complex reasoning capabilities. We selected prompts from LongBench (Bai et al., 2024a), ZeroSCROLLS (Shaham et al., 2023), and IFEval (Zhou et al., 2023). For long-sequence tasks, we used the Qwen2.5 (0.5B-7B) (Yang et al., 2025b) and Llama3.2 (1B, 3B) (Dubey et al., 2024) model series to generate outputs. For reasoning tasks, outputs were generated by the Qwen2.5 and DeepSeek-R1-Distill model series (Guo et al., 2025).

Scenario 2: Dynamic RL Sampling. This scenario is designed to simulate the dynamic sampling process in RL training. We collected data from the actual GRPO training pipeline of the Qwen2.5 and Llama3.2

model series. Prompts were sourced from six widely-used math and code reasoning datasets: CRUXEval (Gu et al., 2024), GSM8K (Cobbe et al., 2021), LiveCodeBench (Jain et al., 2025), MATH (Hendrycks et al., 2021b), MBPP (Austin et al., 2021), and MMLU-STEM (Hendrycks et al., 2021a). For each prompt, we applied a grouped sampling strategy with K=4 and recorded generated candidate responses and lengths.

Data Splits and Statistics. To ensure a fair evaluation, we strictly adhere to the official train/val/test splits of the source datasets. This guarantees that prompts in the validation and test sets are unseen during the predictor's training phase. Detailed statistics of the dataset are presented in Appendix Table 4.

4.2 Experiment Setting

Baselines and Additional Datasets. We compare our method against baselines including SSJF-Reg (Qiu et al., 2024), SSJF-MC (Qiu et al., 2024), S3 (Jin et al., 2023), PiA (Zheng et al., 2023), TPV (Eisenstadt et al., 2025), TRAIL (Shahout et al., 2025), and LTR-C (Fu et al., 2024). Details about these baselines are shown in Appendix E.1. Our evaluation is conducted on the popular LMSYS (Zheng et al., 2024) benchmark, as well as our ForeLen benchmark, which is designed to be richer and more challenging.

Evaluation Metrics. We use the Mean Absolute Error (MAE) to evaluate the performance of our method. Use throughput, Job Completion Time and padding ratio to evaluate the end-to-end system performance. Details are shown in Appendix B.

Experimental Setup. For training, we use the AdamW (Kingma & Ba, 2015; Loshchilov & Hutter, 2019) optimizer with a learning rate of **2e-5**. We train the model for a maximum of 10 epochs with a batch size of 16. For reproducibility across all experiments, we set the random seed to 42. For the Soft Label Regression specific settings, the target length is discretized into $K = 20$ bins. And we set λ to 0.95 to balance the CE loss and MSE loss. Experiments are run with 1 V100 GPU, 10 core CPU, and 64 GB memory. For all baseline methods, we adopt the hyperparameter settings reported in their original papers.

4.3 Main Results: Prediction Accuracy

To evaluate the efficacy of our proposed method, EGTP, we conducted a comprehensive comparison against a suite of state-of-the-art baselines for output length prediction. As presented in Table 1, our evaluation measures the MAE on two distinct benchmarks: the widely-used LMSYS dataset and our more challenging ForeLen benchmark. The results unequivocally demonstrate that EGTP consistently and significantly outperforms all other methods across every model and scenario. On the standard LMSYS benchmark, EGTP achieves the lowest MAE when predicting output lengths for both GPT-4 (87.32) and Claude-2 (68.33), surpassing the next-best methods by 9.1% and 11.3%, respectively. This initial result validates the fundamental effectiveness of our approach on established, real-world conversational data.

We further assessed our method on the more demanding ForeLen benchmark, which incorporates complex scenarios involving Long Sequences, Reasoning, and data from LLMs Reinforcement Learning. Even in these challenging conditions, EGTP maintains its exceptional performance and reaffirms its superiority. When analyzing the average performance across these scenarios, EGTP establishes a substantial margin over the strongest baseline, TRAIL, reducing the MAE from 146.42 to 110.75 (a 24.4% improvement) for Qwen2.5 3B; from 137.96 to 103.47 (a 25.0% improvement) for Qwen2.5 7B; from 151.80 to 105.08 (a 30.8% improvement) for Llama3.2 1B; and from 157.88 to 101.53 (a remarkable 35.7% improvement) for Llama3.2 3B. This consistent outperformance across diverse models and complex tasks highlights the excellent generalization capabilities of EGTP and strongly underscores the critical role of token entropy in providing a robust signal for accurate output length prediction.

The Effect of PLP. The experimental results in Figure 3 for Progressive Length Prediction show consistent improvements across all three tasks. The RL task demonstrates the strongest performance gains, dropping

Model	Scenario	Prediction Method							
		EGTP(Ours)	SSJF-Reg	SSJF-MC	S3	PiA	TPV	TRAIL	LTR-C
LMSYS Benchmark									
GPT-4	LMSYS	**87.32**	171.62	190.93	**96.03**	143.02	339.88	116.91	104.11
Claude-2	LMSYS	**68.33**	152.00	140.18	83.51	91.32	283.81	102.39	**77.03**
ForeLen Benchmark									
Qwen2.5 3B	LongSeq	**93.43**	271.15	771.96	186.82	346.36	576.08	147.92	**124.23**
	Reasoning	**139.04**	325.38	789.21	169.72	428.10	621.72	**132.20**	145.53
	RL	**99.78**	194.03	187.04	169.306	197.87	238.20	**159.15**	192.84
	Avg.	**110.75**	263.52	582.74	175.28	324.11	478.67	**146.42**	154.20
Qwen2.5 7B	Long Seq	**81.60**	210.68	507.34	161.82	279.55	533.92	134.18	**129.37**
	Reasoning	**133.57**	298.80	770.92	168.80	412.84	466.56	**124.19**	134.55
	RL	**95.24**	177.07	167.18	173.37	212.78	202.98	**155.51**	187.13
	Avg.	**103.47**	228.85	481.81	168.00	301.72	401.15	**137.96**	150.35
Llama3.2 1B	Long Seq	**81.77**	173.97	251.71	262.27	145.61	512.16	**145.35**	179.11
	Reasoning	**138.04**	157.67	143.78	152.29	254.57	273.04	148.28	**142.28**
	RL	**95.44**	204.87	267.08	235.07	197.87	308.09	**161.78**	206.70
	Avg.	**105.08**	178.84	220.86	216.54	199.35	364.43	**151.80**	176.03
Llama3.2 3B	Long Seq	**78.83**	193.34	234.83	259.55	149.92	733.75	**143.62**	317.89
	Reasoning	**111.23**	186.16	**160.36**	164.22	264.66	373.03	177.16	174.01
	RL	**114.53**	418.09	218.00	163.86	197.87	244.99	152.85	**131.75**
	Avg.	**101.53**	265.86	204.40	195.88	204.15	450.59	**157.88**	207.88

Table 1: Comparison of different length prediction methods. (**Best**,**SecondBest**)

from 95.24 to 80.85 MAE, while both Reasoning and Long Seq tasks also show substantial improvements. All tasks exhibit a similar convergence pattern where rapid initial gains in the first few steps gradually stabilize, suggesting that PLP effectively leverages already-generated tokens to refine length predictions. The improvement across diverse task types validates PLP's core approach of progressive refinement during the decoding process, making it particularly valuable for dynamic resource allocation scenarios where accurate length prediction is crucial for efficient scheduling.

4.4 End-to-End System Performance

To evaluate the practical effectiveness of our proposed EGTP, we integrated it and baseline predictors with a Shortest Job First (SJF) scheduler in an end-to-end system powered by the vLLM serving backend. We tested the system on two distinct workloads: Long Sequence and Reasoning. The results, presented in the Table 2, unequivocally demonstrate that EGTP consistently and substantially outperforms all baselines across every metric in both scenarios. In the Long Sequence workload, EGTP not only achieves the highest throughput but also more than halves the JCT compared to the best-performing baseline, TRAIL. This significant performance gain is directly driven by EGTP's superior prediction accuracy, which slashes the padding ratio to just 0.18, a nearly 3x reduction over TRAIL's 0.51. This trend extends to the Reasoning scenario, where EGTP again leads all methods by reducing the padding ratio to a mere 0.09. This represents a 36% relative reduction in wasted computation over the strongest baseline, LTR-C, cementing its lead in throughput and JCT.

4.5 Ablation Study

Effectiveness of Entropy-Guided Pooling. Our ablation study on different pooling methods, with results